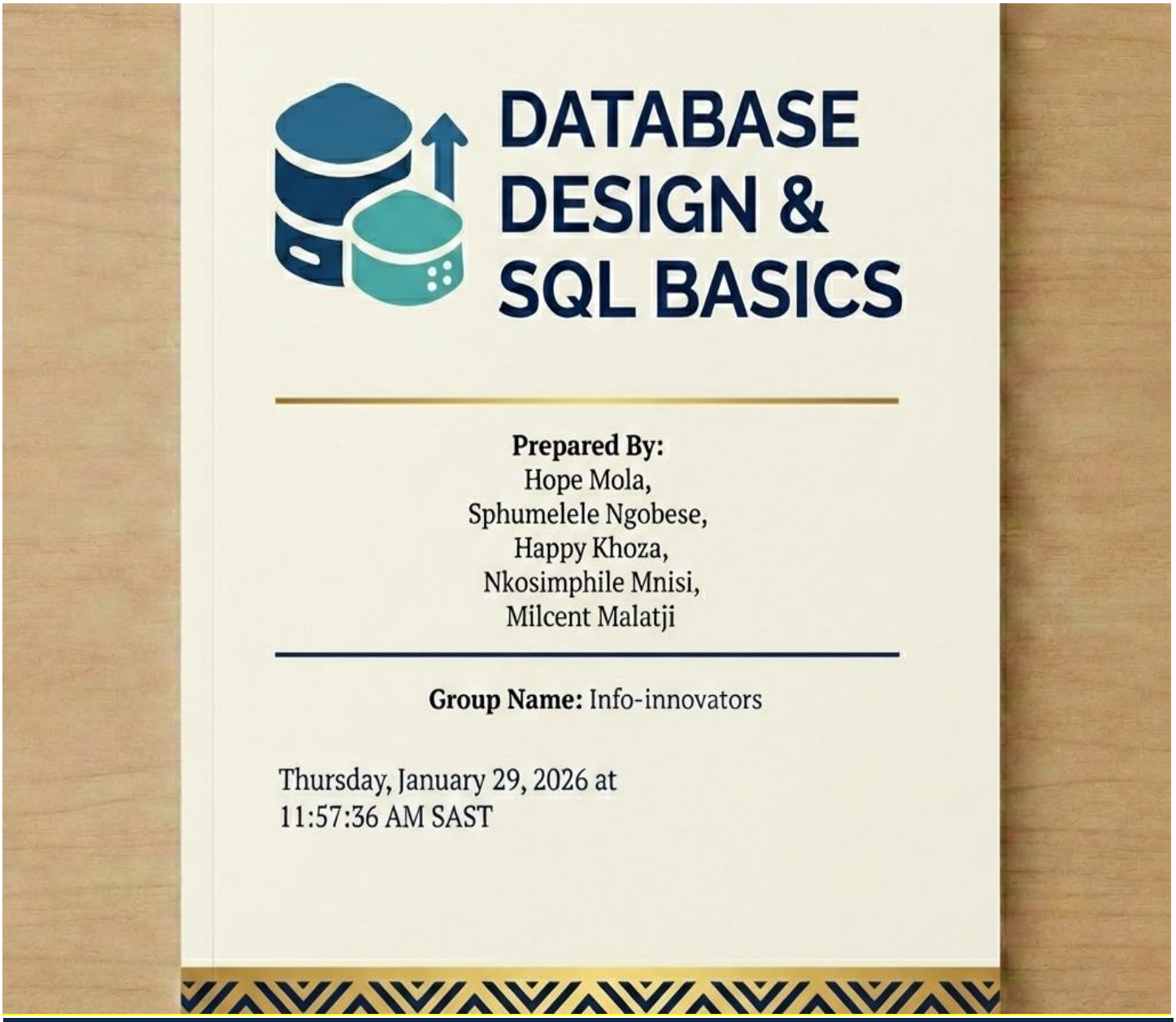


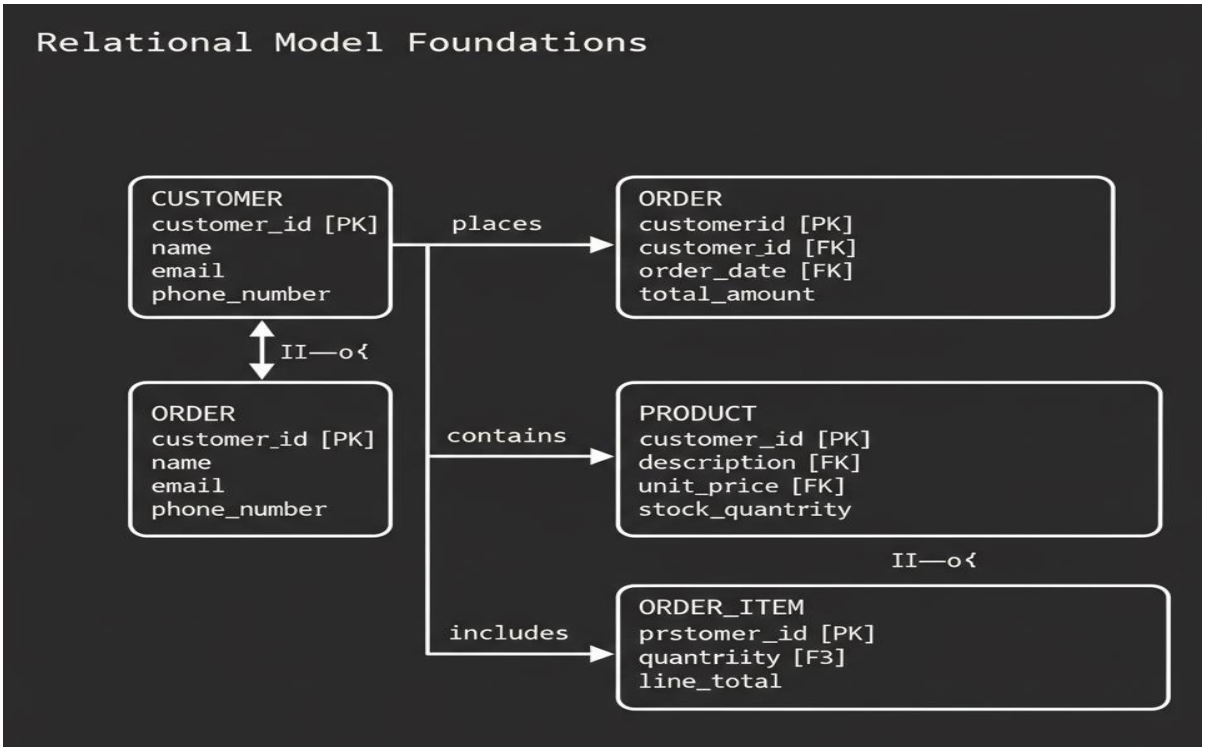
Database Design & SQL Basics

Research

Q1. Relational Model Foundations

The **relational model**, introduced by E.F. Codd in 1970, organizes data into **relations** (tables), where each relation is a set of **tuples** (rows). Each tuple consists of **attributes** (columns), each drawn from a **domain** that defines permissible values (data type and constraints). Relations must satisfy the property that tuples are unordered and unique, and attributes are unordered but uniquely named within a relation.

A **domain** ensures data integrity by restricting attribute values—for example, a Salary domain may only allow positive integers. **Logical data independence** is a core principle: it allows changes to the logical schema (e.g., adding columns) without requiring modifications to application code. This is achieved through the separation of the conceptual/logical layer from the physical storage layer, enabling database evolution without disrupting existing queries. In modern systems, this supports scalability, maintainability, and adaptability to changing business requirements (Elmasri & Navathe, 2016).



Q2. Keys & Constraints in Design Quality

Keys uniquely identify tuples:

- **Candidate keys** are minimal sets of attributes that uniquely identify a tuple.
- **Primary keys** are chosen candidate keys for table identification.
- **Surrogate keys** are system-generated (e.g., auto-increment IDs) and have no business meaning, simplifying joins and insulation from business logic changes.
- **Foreign keys** enforce referential integrity by linking to a primary key in another table.

Constraints prevent anomalies:

- **NOT NULL** ensures mandatory data.
- **UNIQUE** enforces candidate key uniqueness.
- **CHECK** validates domain rules (e.g., Salary > 0).
- **Referential constraints** (foreign keys) prevent orphan records via CASCADE, SET NULL, or RESTRICT actions.

These constraints combat **insertion**, **update**, and **deletion anomalies** by preserving entity and referential integrity (Silberschatz, Korth, & Sudarshan, 2020).

Q3. Normalization vs. Denormalization

Normalization eliminates redundancy and anomalies:

- **1NF**: Atomic values, no repeating groups.
- **2NF**: No partial dependency on a composite primary key.
- **3NF**: No transitive dependency (non-key attributes depend only on the key).

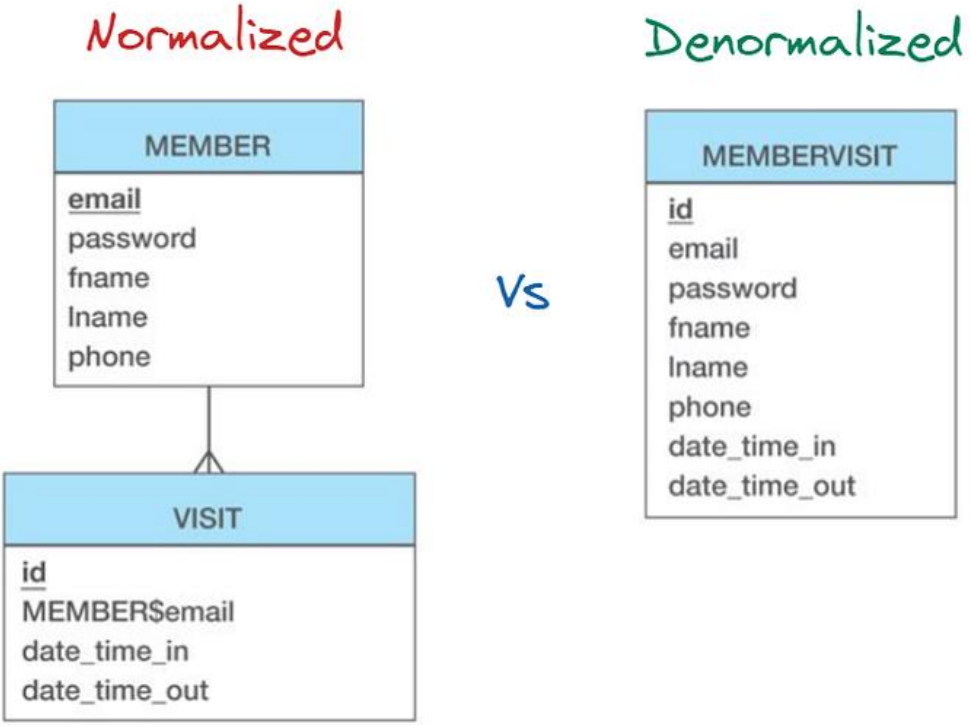
- **BCNF**: Every determinant is a candidate key (stronger than 3NF).

Normalization prevents **update anomalies** (inconsistencies from modifying redundant data) but may increase join complexity.

Denormalization intentionally reintroduces redundancy for performance gains, especially in **analytical systems** (OLAP) where read-heavy queries benefit from fewer joins. It is justified for:

- Reporting and data warehousing.
- Optimizing frequent complex joins.
- Caching aggregated values (e.g., storing OrderTotal instead of summing OrderItems each time).

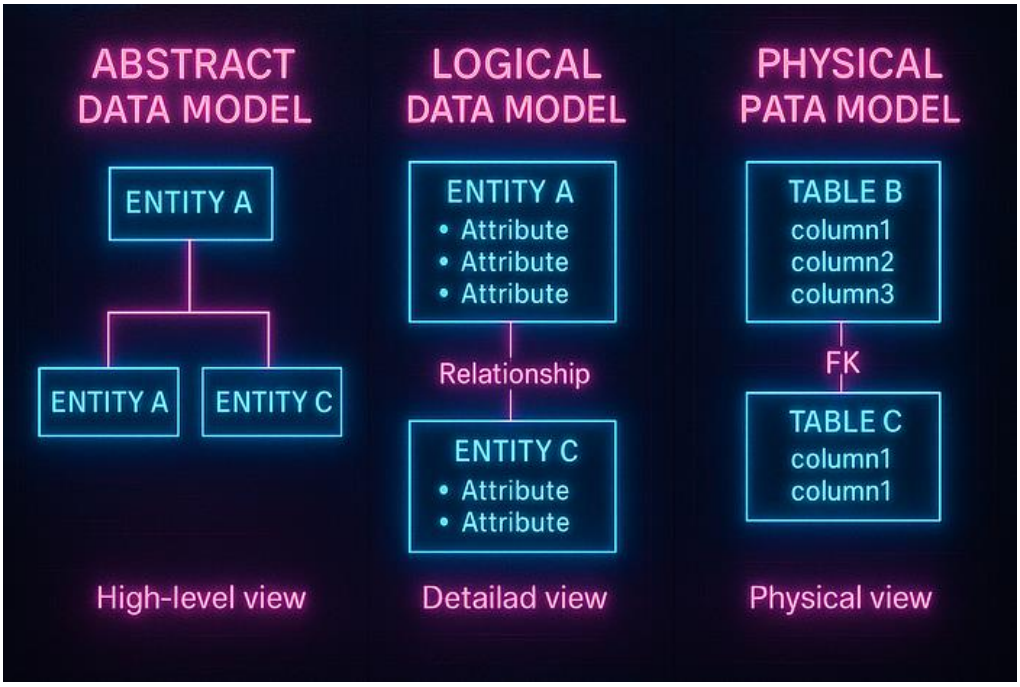
Trade-offs include potential data inconsistency and increased storage (Garcia-Molina, Ullman, & Widom, 2009).



Q4. Conceptual vs Logical vs Physical Design

- **Conceptual design** uses **ER modeling** (entities, relationships, cardinalities) to capture business requirements abstractly, independent of DBMS specifics.
- **Logical design** translates ER diagrams into **relational schemas**: entities become tables, attributes become columns, relationships become foreign keys. Normalization is applied here.
- **Physical design** implements the logical schema in a specific DBMS, adding **indexes**, partitioning, storage parameters, and access paths for performance.

For example, an ER “Customer” entity with attributes becomes a Customers table. Physically, a B-tree index on CustomerID speeds up lookups. This layered approach ensures clarity, maintainability, and performance tuning flexibility (Connolly & Begg, 2015).



Q5. JOIN Semantics & Join Strategies

JOIN types:

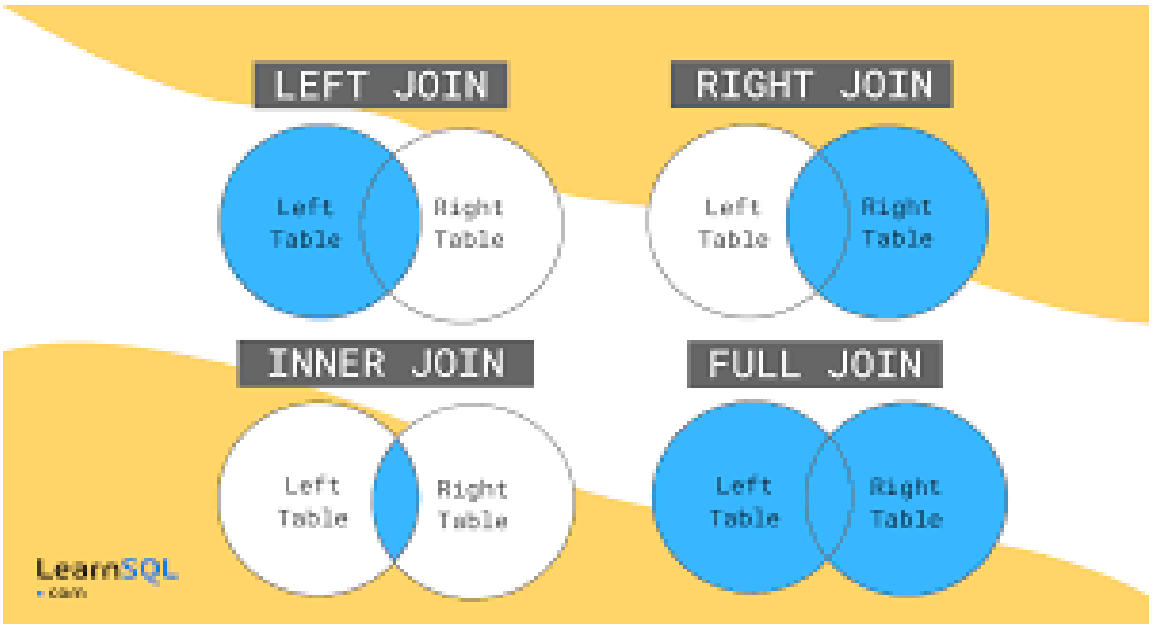
- **INNER JOIN**: Returns matching rows from both tables.
- **LEFT/RIGHT OUTER JOIN**: Returns all rows from one table, matched rows from the other, NULLs for non-matches.
- **FULL OUTER JOIN**: Returns all rows from both tables, NULLs for non-matches.
- **CROSS JOIN**: Cartesian product of all rows.

NULLs propagate in outer joins and are treated as unequal in comparisons.

Join algorithms:

- **Nested Loop**: Efficient for small tables or indexed joins.
- **Hash Join**: Good for large, unsorted tables; builds a hash table on one side.
- **Merge Join**: Requires sorted inputs; efficient for large sorted datasets.

Optimizers choose based on table size, indexes, memory, and data distribution (Oracle Database SQL Tuning Guide, 2023).



Q6. GROUP BY, Aggregation & Set Semantics

GROUP BY partitions rows into groups by specified columns. **Aggregate functions** (SUM, COUNT, AVG, etc.) compute per-group values. **HAVING** filters groups post-aggregation (unlike WHERE, which filters rows pre-aggregation).

COUNT(*) vs COUNT(column):

- COUNT(*) counts all rows in a group.
- COUNT(column) counts non-NULL values in that column.

NULLs in aggregation: Ignored in SUM, AVG, COUNT(column); GROUP BY treats NULLs as a distinct group.

Pitfalls:

- Selecting non-aggregated columns not in GROUP BY causes ambiguity (invalid in standard SQL).
- Misplacing filter conditions (WHERE vs HAVING).
- Overlooking NULL groups (PostgreSQL Documentation, 2023)

Q7. Transactions, Isolation & Integrity

ACID properties:

- **Atomicity**: All-or-nothing execution.
- **Consistency**: Transactions move the database between valid states.
- **Isolation**: Concurrent transactions do not interfere.
- **Durability**: Committed changes survive failures.

Isolation levels (from least to most strict):

- **READ UNCOMMITTED**: Allows dirty reads.
- **READ COMMITTED**: Prevents dirty reads.
- **REPEATABLE READ**: Prevents non-repeatable reads.

- **SERIALIZABLE**: Prevents phantom reads.

Anomalies:

- **Dirty read**: Reading uncommitted data.
- **Non-repeatable read**: Different values on re-read.
- **Phantom read**: New rows appear in a re-executed query.

Constraints (e.g., foreign keys) are checked at transaction commit, ensuring integrity even with concurrent modifications (Bernstein & Newcomer, 2009).

Q8. Indexing for Query Performance

B-tree indexes (balanced tree) support:

- Equality and range predicates (=, >, BETWEEN).
- Join operations (speeding up nested loop or merge joins).
- ORDER BY and GROUP BY (providing sorted access).

Alternative indexes:

- **Hash indexes**: Fast equality lookups, no sorting.
- **Bitmap indexes**: Low-cardinality columns in data warehouses.

Risks:

- Write overhead (indexes slow INSERT, UPDATE, DELETE).
- Storage cost.
- Poor selectivity (indexing low-distinct columns may be ineffective).

Interaction with GROUP BY and JOIN plans: Indexes can enable **index-only scans** (avoiding table access) and influence optimizer choices for join order and

algorithm. For example, a B-tree index on CustomerID can turn a hash join into a faster nested loop join with index lookup (MySQL 8.0 Reference Manual, 2023).

References

1. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.
3. Garcia-Molina, H., Ullman, J. D., & Widom, J. (2009). *Database Systems: The Complete Book* (2nd ed.). Prentice Hall.
4. Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson.
5. Oracle Database SQL Tuning Guide, 12c Release 2 (2023). Oracle Help Center.
6. PostgreSQL 15.2 Documentation (2023). The PostgreSQL Global Development Group.
7. Bernstein, P. A., & Newcomer, E. (2009). *Principles of Transaction Processing* (2nd ed.). Morgan Kaufmann.
8. MySQL 8.0 Reference Manual (2023). Oracle Corporation.